

C语言指针与链表专项训练 · Day 2

参考：《C程序设计》（谭浩强）

题型：单项选择题

题量：50题

重点：指针、动态内存、单链表；综合函数、循环和输入输出

答题说明

- 每题只有一个正确答案。
- 建议独立完成，不查阅资料。
- 将答案写在文末答题卡中。
- 把握不准的题请加 ? ，例如： 12.B? 。即使答对，批改时也会详细讲解。

一、指针基础与函数参数

1. 执行下列代码后，变量 a 的值是：

```
int a = 7;  
int *p = &a;  
*p = 12;
```

- A. 7
- B. 12
- C. a 的地址
- D. 不确定

2. 已定义 `int a = 10;`，下列初始化指针 p 的语句正确的是：

- A. `int *p = a;`
- B. `int *p = &a;`
- C. `int p = &a;`
- D. `int *p = *a;`

3. 下列程序的输出是：

```
int a[] = {2, 4, 6, 8};  
int *p = a;  
p += 2;  
printf("%d", *p);
```

- A. 2
- B. 4
- C. 6
- D. 8

4. 已定义 `int a[5];`，表达式 `sizeof(a) / sizeof(a[0])` 的值是：

- A. 1
- B. 4
- C. 5
- D. 指针所占字节数

5. 对于下面的函数形参，函数体中的 `sizeof(a)` 等价于：

```
void fun(int a[])  
{  
    /* ... */  
}
```

- A. `sizeof(int) * 实参数组元素个数`
- B. `sizeof(a[0])`
- C. `sizeof(int *)`
- D. 0

6. 执行下列代码后， x 的值是：

```
void add3(int *p)
{
    *p += 3;
}

int x = 5;
add3(&x);
```

- A. 3
- B. 5
- C. 8
- D. 不确定

7. 已定义 `const int *p;`，下列说法正确的是：

- A. 不能改变 p 的指向，也不能通过 p 修改所指对象
- B. 可以改变 p 的指向，但不能通过 p 修改所指对象
- C. 不能改变 p 的指向，但可以通过 p 修改所指对象
- D. p 和 *p 都可以任意修改

8. 已定义 `int *const p = &a;`，下列说法正确的是：

- A. 可以改变 p 的指向，但不能修改 *p
- B. 不能改变 p 的指向，但可以修改 *p
- C. p 和 *p 都不能修改
- D. p 是指向常量的普通指针

9. 执行下列代码后， a 的值是：

```
int a = 3;
int *p = &a;
int **pp = &p;
**pp = 9;
```

- A. 3
- B. 9
- C. p 的地址

- D. 不确定

10. 下列语句声明了一个“指向含4个整型元素的一维数组”的指针：

- A. `int *p[4];`
- B. `int (*p)[4];`
- C. `int **p[4];`
- D. `int p*[4];`

11. 对二维数组 `int a[3][4];`，表达式 `a[2][3]` 等价于：

- A. `*(a + 2 + 3)`
- B. `*(*(a + 2) + 3)`
- C. `**a + 2 + 3`
- D. `*(a[2]) + 3`

12. 已定义：

```
int a[3][4];  
int (*p)[4] = a;
```

则表达式 `*(p + 1) + 1` 指向：

- A. `a[0][3]`
- B. `a[1][0]`
- C. `a[1][1]`
- D. `a[2][1]`

二、字符指针与动态内存

13. 执行下列定义后，`sizeof(s)` 的值是：

```
char s[] = "abc";
```

- A. 3
- B. 4

- C. 指针所占字节数
- D. 不确定

14. 对下列代码的描述正确的是：

```
char *s = "abc";  
s[0] = 'A';
```

- A. 一定得到字符串 "Abc"
- B. 一定编译失败
- C. 试图修改字符串字面量，行为未定义
- D. s 会自动变成字符数组

15. 下列函数的作用是：

```
int fun(const char *s)  
{  
    const char *p = s;  
    while (*p != '\0')  
        p++;  
    return (int)(p - s);  
}
```

- A. 复制字符串
- B. 比较两个字符串
- C. 计算字符串长度
- D. 逆置字符串

16. 自己编写字符串复制程序时，复制完所有普通字符后还必须：

- A. 在目标字符串末尾写入 `\0`
- B. 释放源字符串
- C. 把目标指针设为 `NULL`
- D. 删除目标字符串的首字符

17. 动态申请 `n` 个 `int` 元素的空间，推荐写法是：

- A. `int *p = malloc(n);`
- B. `int *p = malloc(sizeof(p));`

- C. `int *p = malloc(n * sizeof *p);`
- D. `int p = malloc(n * sizeof(int));`

18. `malloc()` 申请内存失败时返回：

- A. -1
- B. 字符 '0'
- C. NULL
- D. 空字符串

19. 执行 `free(p);` 后，为减少误用悬空指针的风险，通常应继续执行：

- A. `p++;`
- B. `*p = 0;`
- C. `p = NULL;`
- D. 再次执行 `free(p);`

20. 使用 `realloc()` 扩大内存时，更安全的写法是：

- A.

```
p = realloc(p, new_size);
```

- B.

```
int *temp = realloc(p, new_size);  
if (temp != NULL)  
    p = temp;
```

- C.

```
free(p);  
realloc(p, new_size);
```

- D.

```
p++;  
realloc(p, new_size);
```

三、单链表基础与基本操作

以下各题若无特别说明，结点类型为：

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

21. 单链表结点中，成员 `next` 的作用是：

- A. 保存当前结点的数据
- B. 保存下一个结点的地址
- C. 保存链表结点总数
- D. 保存头指针的地址

22. 对不带头结点的单链表，表示空表的条件是：

- A. `head == NULL`
- B. `head->next == NULL`
- C. `head->data == 0`
- D. `head->next == head`

23. 依次输入 1、2、3，每次都使用头插法插入不带头结点的链表，最终顺序是：

- A. 1 2 3
- B. 3 2 1
- C. 1 3 2
- D. 2 3 1

24. 依次输入 1、2、3，使用尾插法建立链表，最终顺序是：

- A. 1 2 3
- B. 3 2 1
- C. 2 1 3
- D. 3 1 2

25. 遍历不带头结点的普通单链表，并访问包括尾结点在内的所有结点，循环条件应为：

- A. while (p->next != NULL)
- B. while (p != NULL)
- C. while (*p != NULL)
- D. while (p == NULL)

26. 已知 p 指向链表中的一个结点，将 newnode 插入到 p 后面的正确顺序是：

- A.

```
p->next = newnode;  
newnode->next = p->next;
```

- B.

```
newnode->next = p->next;  
p->next = newnode;
```

- C.

```
newnode = p->next;  
p->next = newnode;
```

- D.

```
p = newnode;  
newnode->next = NULL;
```

27. 已知 `p->next` 是待删除结点，正确的删除代码是：

- A.

```
free(p->next);  
p->next = p->next->next;
```

- B.

```
Node *q = p->next;  
p->next = q->next;  
free(q);
```

- C.

```
p->next = NULL;  
free(p);
```

- D.

```
free(p);  
p = p->next;
```

28. 函数参数为 `Node **head`，且 `*head != NULL` 时，删除不带头结点链表的首结点，正确代码是：

- A.

```
Node *q = *head;  
*head = q->next;  
free(q);
```

- B.

```
free(*head);  
*head = (*head)->next;
```

- C.

```
*head = NULL;
```

- D.

```
free(head);
```

29. 删除首结点的函数使用 Node **head ， 主要是为了：

- A. 修改主调函数中的头指针
- B. 让结点占用更少内存
- C. 避免调用 free()
- D. 把链表变成数组

30. 下列代码中，若 head 是不带头结点链表的头指针，变量 count 最终表示：

```
int count = 0;
Node *p = head;
while (p != NULL) {
    count++;
    p = p->next;
}
```

- A. 链表中数据为0的结点数
- B. 链表中的结点总数
- C. 链表最后一个结点的数据
- D. 头结点的地址

31. 在链表中查找第一个数据域等于 x 的结点，正确的核心循环是：

- A.

```
while (p != NULL && p->data != x)
    p = p->next;
```

- B.

```
while (p == NULL && p->data == x)
    p++;
```

- C.

```
while (p->next == p)
    free(p);
```

- D.

```
while (p != NULL)
    head = NULL;
```

32. 原地逆置单链表时，正确的核心操作顺序是：

- A.

```
next = p->next;
p->next = previous;
previous = p;
p = next;
```

- B.

```
p = p->next;
p->next = previous;
previous = p;
```

- C.

```
free(p);
p = next;
```

- D.

```
previous->next = p;
p = p->next;
```

33. 使用三个指针逆置链表时，开始循环前通常应设置：

- A. `previous = NULL; p = head;`
- B. `previous = head; p = NULL;`
- C. `previous = head->next; p = head;`
- D. `previous = p = NULL;`

34. 一个非空单链表完成逆置后，原来的首结点将成为：

- A. 新链表的首结点
- B. 新链表的尾结点，且其 `next` 应为 `NULL`
- C. 头结点之前的结点
- D. 被自动释放的结点

35. 使用快慢指针判断链表是否存在环时，通常：

- A. 快指针每次走一步，慢指针每次走两步
- B. 快指针每次走两步，慢指针每次走一步
- C. 两个指针始终不移动
- D. 两个指针都只移动一次

36. 快慢指针按各自步长移动后再次出现 `slow == fast`，表示：

- A. 链表有环
- B. 链表为空
- C. 两个指针都指向尾结点的后继
- D. 链表已经被逆置

37. 合并两个按升序排列的单链表时，正确的基本方法是：

- A. 比较当前结点的数据，将较小结点接到结果链表后
- B. 每次删除两个链表的首结点
- C. 只连接两个链表的尾结点，不比较数据
- D. 把所有数据都修改为相同值

38. 用递归方式逆置链表时，合理的递归终止条件是：

- A. `head == NULL || head->next == NULL`
- B. `head->data == 0`
- C. `head->next == head`，且只适用于普通单链表

- D. head != NULL && head->next != NULL

39. 释放整条单链表时，循环中正确的顺序是：

- A. 先保存当前结点的后继，再释放当前结点
- B. 先释放当前结点，再通过当前结点访问后继
- C. 只执行 head = NULL ，不调用 free()
- D. 只释放尾结点

40. 对带头结点的单链表，若 head 指向头结点，则空表条件通常是：

- A. head == NULL
- B. head->next == NULL
- C. head->data == 0
- D. head->next == head

四、函数、循环与输入输出

41. 下列函数声明中，表示函数 area 接收一个 double 参数并返回 double 的是：

- A. void area(double);
- B. double area(double);
- C. double *area;
- D. area(double) double;

42. 执行下列代码后，a 的值是：

```
void fun(int x)
{
    x = 20;
}

int a = 10;
fun(a);
```

- A. 10
- B. 20
- C. 0
- D. 不确定

43. 递归计算非负整数阶乘时，下列终止条件最合适的是：

- A. if (n < 0) return 0;
- B. if (n > 1) return 1;
- C. if (n == 0 || n == 1) return 1;
- D. 不需要终止条件

44. 执行下列代码，输出是：

```
int fun(void)
{
    static int x = 0;
    return ++x;
}

int a = fun();
int b = fun();
printf("%d %d", a, b);
```

- A. 1 1
- B. 1 2
- C. 2 2
- D. 0 1

45. 执行下列循环后， sum 的值是：

```
int i, sum = 0;
for (i = 1; i <= 5; i++)
    sum += i;
```

- A. 5
- B. 10
- C. 15
- D. 20

46. 执行下列代码后， count 的值是：

```
int i = 5, count = 0;
do {
    count++;
    i++;
} while (i < 3);
```

- A. 1
- B. 0
- C. 3
- D. 无限循环

47. 执行下列代码后， sum 的值是：

```
int i, sum = 0;
for (i = 1; i <= 5; i++) {
    if (i % 2 == 0)
        continue;
    sum += i;
}
```

- A. 6
- B. 8
- C. 9
- D. 15

48. 已定义 int n; ，从键盘输入一个整数并存入 n ，正确语句是：

- A. scanf("%d", n);
- B. scanf("%d", &n);
- C. scanf("%f", &n);
- D. printf("%d", &n);

49. 已定义 char s[20]; ，下列语句能够较安全地读入一个不含空白字符、长度不超过19的字符串的是：

- A. scanf("%s", &s);
- B. scanf("%19s", s);

- C. `scanf("%20c", s);`
- D. `printf("%19s", s);`

50. 从键盘读取一个字符并立即输出该字符，正确代码是：

- A.

```
char ch;  
printf("%c", ch);
```

- B.

```
int ch = getchar();  
putchar(ch);
```

- C.

```
int ch;  
scanf("%s", &ch);
```

- D.

```
getchar(ch);  
putchar();
```

答题卡

1. __	2. __	3. __	4. __	5. __
6. __	7. __	8. __	9. __	10. __
11. __	12. __	13. __	14. __	15. __
16. __	17. __	18. __	19. __	20. __
21. __	22. __	23. __	24. __	25. __
26. __	27. __	28. __	29. __	30. __
31. __	32. __	33. __	34. __	35. __
36. __	37. __	38. __	39. __	40. __
41. __	42. __	43. __	44. __	45. __
46. __	47. __	48. __	49. __	50. __

做完后将填写答案的文件或清晰照片发给我批改。